Slide 1: Title and Course Material

This presentation is based on the 10th Edition of the textbook "Computer Organization and Architecture" by William Stallings. The entire focus of this section is to understand the physical and logical realities of how computer hardware performs under heavy workloads.

Slide 2: Chapter 2 - Performance Issues

Chapter 2 focuses entirely on "Performance Issues". This means analyzing how engineers continuously push for faster computers, what physical walls they hit when making microchips, and the clever architectural tricks used to bypass those physical limitations.

Slide 3: Designing for Performance

The computing industry is locked in a relentless cycle where the cost of computer systems continues to drop dramatically, while the performance and capacity of those systems rise equally dramatically.

- **The Mainframe Comparison:** To put this exponential growth into perspective, the standard laptop sitting on your desk today possesses the same raw computing power as a massive, room-sized IBM mainframe from just 10 or 15 years ago.
- **Disposable Power:** Processors have become so inexpensive and ubiquitous that we now have microprocessors that we essentially throw away—think of the chips in musical greeting cards or smart packaging.
- **Why Do We Need So Much Power?:** Software engineers continually invent applications that eat up whatever performance hardware engineers create. Desktop applications that demand this immense power include:
    - **Image processing and 3D rendering:** Calculating light bounces and pixel colors for millions of pixels 60 times a second (like in modern video games or architectural software).
    - **Speech recognition and Videoconferencing:** Processing analog audio/video signals into digital data in real-time without noticeable lag.
    - **Multimedia authoring and file annotation:** Editing 4K video or adding voice overlays requires moving gigabytes of data rapidly.
    - **Simulation modeling:** Predicting weather patterns or simulating fluid dynamics requires billions of mathematical calculations.
- **Enterprise and Cloud Demands:** It isn't just desktop users driving this. Businesses rely on incredibly powerful servers to handle massive transaction volumes (like processing thousands of credit card swipes per second) and to run massive database systems. These server banks have completely replaced the old centralized mainframe computer centers of the past. Cloud service providers also rely on massive, high-performance banks of servers to satisfy the needs of countless clients globally.

Slide 4: Microprocessor Speed (The "Tricks" to Go Faster)

You can only crank up the clock speed of a processor (the GHz) so much before it melts. Therefore, engineers build clever techniques into contemporary processors to execute tasks faster without just raising the clock rate:

- **Pipelining:** The processor moves data or instructions into a conceptual "pipe," allowing all stages of the pipe to process simultaneously.
    - *Analogy:* Think of doing laundry. Instead of waiting for one load to go through the washer, dryer, and folding before starting the next load, pipelining lets you put load 2

in the washer while load 1 is in the dryer. Multiple instructions are in different phases of completion at the exact same time.

- **Branch Prediction:** The processor looks ahead in the instruction code fetched from memory and actively predicts which branches (or groups of instructions) are likely to be processed next. If a program has an "if/then" statement, the processor guesses which path will be taken and starts loading it early.
- **Superscalar Execution:** This is the ability of the processor to issue more than one instruction in every single processor clock cycle. In effect, multiple parallel pipelines are used side-by-side. If pipelining is an assembly line, superscalar execution means building three assembly lines next to each other.
- **Data Flow Analysis:** The processor analyzes which instructions are dependent on each other's results, or data, to create a highly optimized schedule of instructions. If Instruction B needs the answer from Instruction A, they must happen in order. But if Instruction C is completely independent, the processor will pull it forward and execute it early to save time.
- **Speculative Execution:** Using branch prediction and data flow analysis, some processors speculatively execute instructions ahead of their actual appearance in the program sequence. The results are held in temporary memory locations. If the processor guessed the branch correctly, it instantly applies the results, keeping the execution engines as busy as possible. If it guessed wrong, it just dumps the temporary data.

Slide 5: Performance Balance

A computer is only as fast as its slowest component. If you build a lightning-fast processor but pair it with slow memory, the processor wastes millions of cycles just waiting for data to arrive. Engineers must adjust the organization and architecture to compensate for the mismatch in capabilities among various components. Architectural examples include:

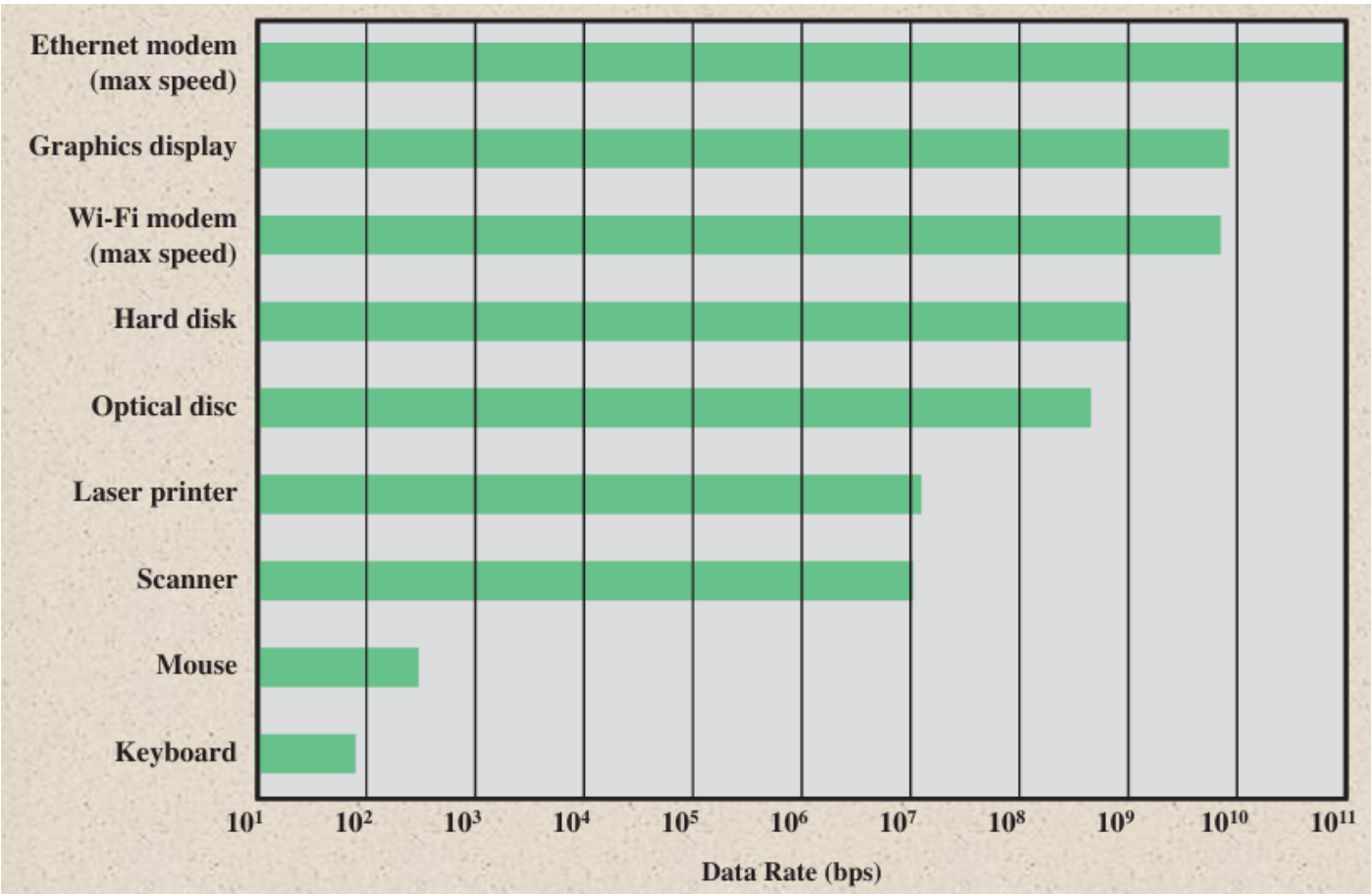
- **Wider Data Paths:** Instead of forcing data through a narrow pipe, engineers increase the number of bits retrieved at one time by making DRAMs (memory) "wider" rather than "deeper," and by using wide bus data paths.
- **Complex Caches:** To reduce the frequency of having to reach all the way out to main memory, engineers incorporate increasingly complex and efficient cache structures directly between the processor and main memory. Cache is ultra-fast, small memory right next to the processor core.
- **DRAM Buffering:** Changing the DRAM interface to make it more efficient by including a cache or other buffering scheme directly on the DRAM chip itself.
- **Higher Speed Interconnects:** Increasing the interconnect bandwidth (the data highways) between processors and memory by using higher speed buses and a hierarchy of buses to efficiently buffer and structure the data flow.

Slide 6: Typical I/O Device Data Rates

This slide presents a chart (Figure 2.1) demonstrating the extreme variance in data rates across different Input/Output devices, measured in bits per second (bps). The scale on the bottom is logarithmic (10^1 to 10^{11}), meaning every major tick mark represents a tenfold increase in speed.

- **Low-Speed Devices (Human Input):** Keyboards operate around 10^2 bps, and mice operate just under 10^3 bps. Because humans physically type and move slowly compared to electronics, these devices need very little bandwidth.
- **Medium-Speed Devices:** Scanners, laser printers, and optical discs require much more data to function, operating in the 10^7 to 10^9 bps range.
- **High-Speed Devices:** Hard disks, Wi-Fi modems, graphics displays, and Ethernet modems demand the most massive data pipelines, operating from 10^9 up to an incredible 10^{11} bps.

The system architecture must accommodate these massive data firehoses without choking the rest of the computer.



Slide 7: Improvements in Chip Organization and Architecture

To keep feeding the demand for speed, chip manufacturers focus on three core areas:

- **Increase the Hardware Speed of the Processor:** This is fundamentally achieved by physically shrinking the size of the logic gates. When gates are smaller, more of them can be packed tightly together, which allows for an increased clock rate. Because the distances between components are shorter, the propagation time for electrical signals is reduced (the electrons literally have less physical distance to travel).
- **Increase the Size and Speed of Caches:** By dedicating a larger portion of the actual processor chip die to cache memory, cache access times drop significantly, keeping the processor constantly supplied with data.
- **Change Processor Organization and Architecture:** This involves fundamentally altering the design to increase the effective speed of instruction execution, primarily through various methods of Parallelism (as discussed in Slide 4 with superscalar execution).

Slide 8: Problems with Clock Speed and Logic Density

Shrinking components and jacking up the clock speed sounds great, but it introduces massive physical limitations that engineers hit in the mid-2000s:

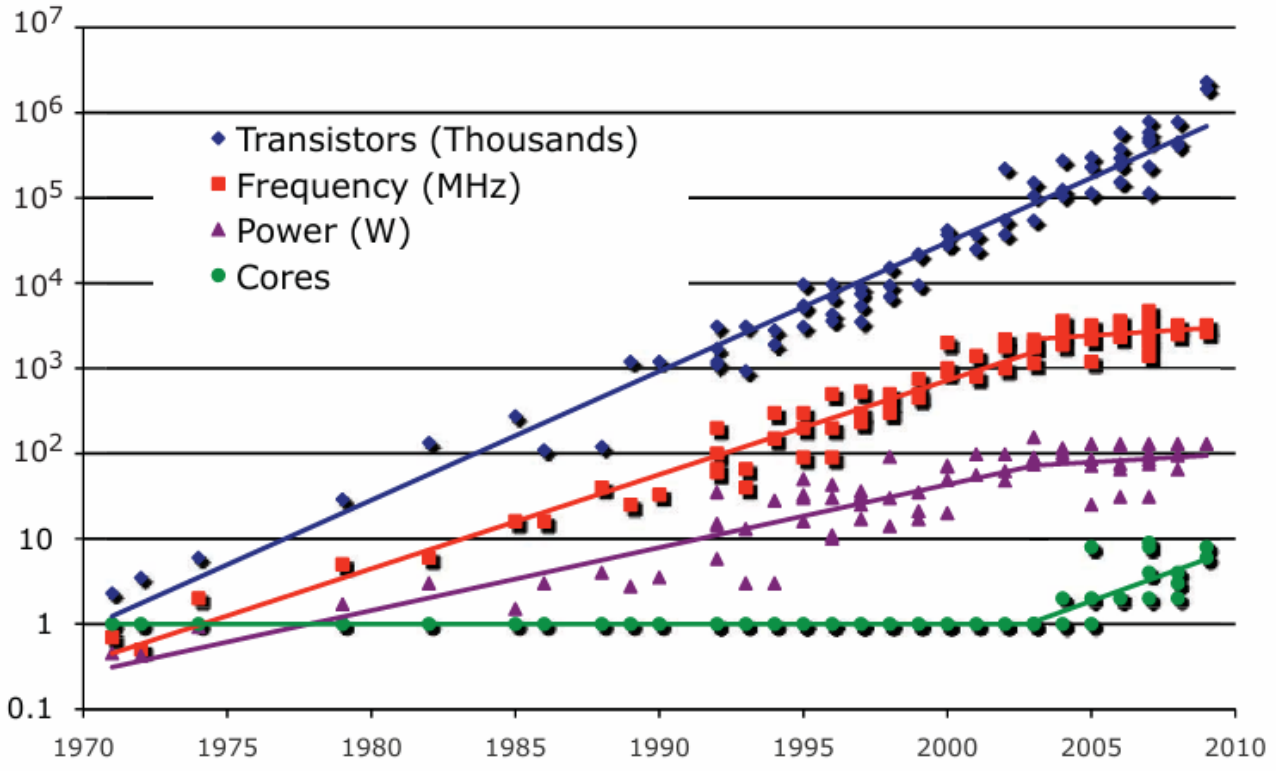
- **Power and Heat:** Power density increases exponentially with the density of the logic gates and the clock speed. Dissipating this heat becomes a critical roadblock. If you run a chip too fast without extreme cooling, it will literally melt itself.
- **RC Delay (Resistance and Capacitance):** The speed at which electrons flow is physically limited by the resistance and capacitance of the microscopic metal wires connecting them.

As components on the chip decrease in size, these wire interconnects become thinner, which heavily increases electrical resistance. Furthermore, to fit everything on the chip, the wires are placed closer together, which increases capacitance (they start interfering with each other's electrical fields). This delay increases as the RC product increases.
- **Memory Latency:** Memory speeds consistently lag behind processor speeds. It takes much longer to fetch data from RAM than it does to process it, creating a perpetual bottleneck.

Slide 9: Processor Trends (Figure 2.2)

This graph perfectly illustrates the physical "wall" the industry hit around the year 2005.

- **Transistors (Blue line):** Continues to climb exponentially in a straight line upward, adhering to Moore's Law. We are still successfully making transistors smaller.
- **Frequency and Power (Red and Purple lines):** Both lines rise steadily from 1970 to 2005, but then they completely flatten out. This is the heat and power wall. Engineers could no longer increase the clock speed (MHz) because they could no longer safely dissipate the required power (W).
- **Cores (Green line):** Notice that from 1970 to 2005, the line for "Cores" is perfectly flat at 1. However, the exact moment the Frequency line flattens out, the Cores line begins to angle upward. Because they couldn't make one core any faster, manufacturers pivoted to putting multiple cores on the same chip.



Slide 10: Multicore Architecture

Faced with the physical impossibility of pushing clock speeds higher, the industry shifted to multicore designs.

- The use of multiple processors on the exact same chip provides the potential to drastically increase overall performance without having to increase the raw clock rate.
- The primary strategy is to use two (or more) simpler processors on the chip rather than trying to engineer one massively complex, overheating processor.

- With two processors sharing the workload, incorporating larger internal caches becomes completely justified.
- As these caches became larger, it made architectural sense to create two, and eventually three, distinct levels of cache (L1, L2, L3) directly on the chip to manage how the different cores share and access data.

Slide 11: Many Integrated Core (MIC) and Graphics Processing Unit (GPU) This slide introduces two highly specialized approaches to parallel processing.

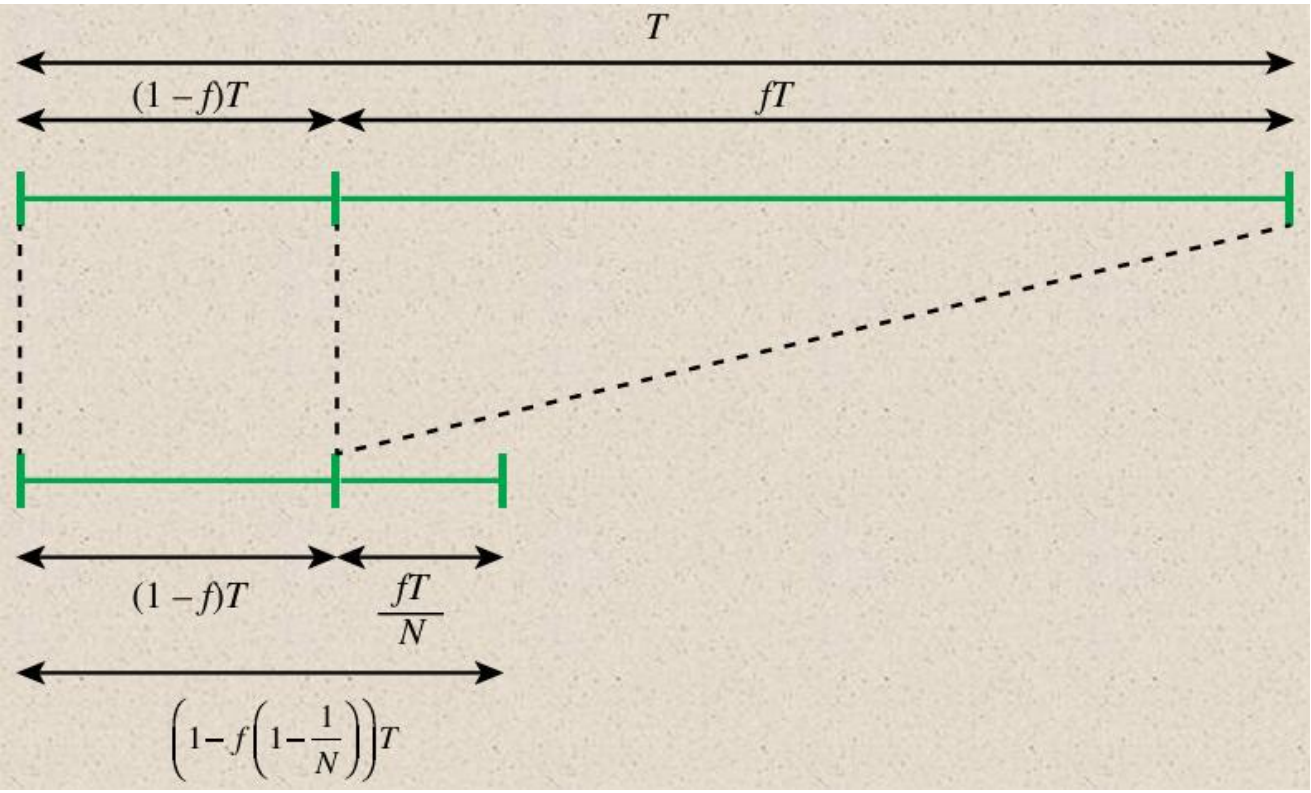
- **MIC (Many Integrated Core):** This architecture represents a massive leap in performance, though it brings significant challenges in writing software that can actually exploit such a huge number of cores. The MIC strategy involves packing a homogeneous (identical) collection of general-purpose processors onto a single chip to handle heavy computational workloads.
- **GPU (Graphics Processing Unit):** A GPU is a specialized core originally designed to perform parallel operations specifically on graphics data. Traditionally found on plug-in graphics cards, GPUs are heavily used to encode and render 2D and 3D graphics, as well as process video. However, because they are so incredibly good at doing the exact same math operation thousands of times simultaneously, they are increasingly used as "vector processors" for a wide variety of non-graphical applications that require highly repetitive computations (like machine learning or scientific simulations).

Slide 12: Amdahl's Law This is a foundational concept in computer science named after Gene Amdahl.

- **The Core Concept:** Amdahl's Law deals with calculating the theoretical potential speedup of a program when you use multiple processors compared to just running it on a single processor.
- **The Reality Check:** It illustrates the fundamental problems facing the computing industry in the development of multi-core machines. Simply adding more cores doesn't magically make everything faster. Software must be specifically adapted to a highly parallel execution environment to actually exploit the power of parallel processing. If a program forces tasks to run sequentially (one after the other), extra cores will just sit idle.
- **Broad Application:** Beyond just processors, Amdahl's Law can be generalized to evaluate and design technical improvements across any computer system.

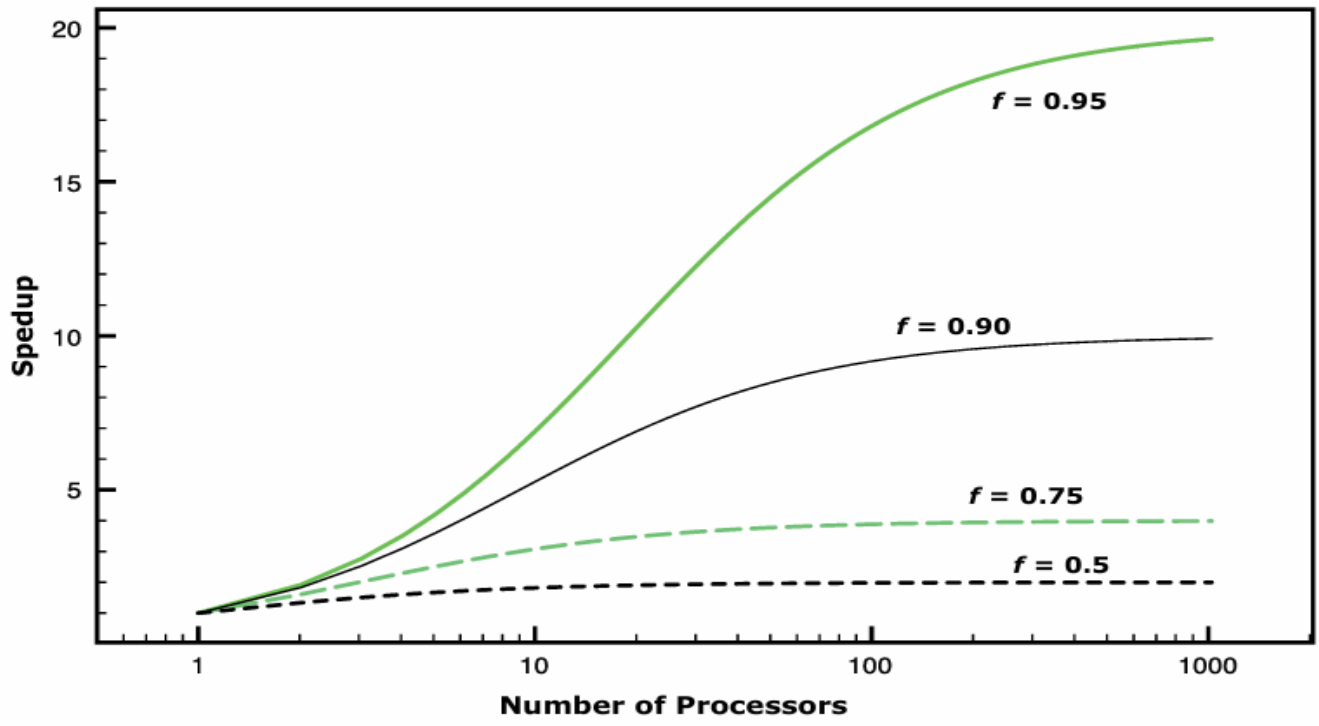
Slide 13: Illustration of Amdahl's Law This slide provides a visual mathematical breakdown of Amdahl's Law (Figure 2.3).

- The top line represents the total execution time (T) of a program on a single processor.
- This time is split into two chunks: the portion of the code that *must* run sequentially ($(1-f)T$) and the portion that can be parallelized (fT).
- The bottom lines show what happens when you add N processors. The sequential portion ($(1-f)T$) takes the exact same amount of time no matter how many cores you add. However, the parallelizable portion shrinks significantly because the work is divided among the cores ($\frac{fT}{N}$). This perfectly illustrates that your maximum speedup is bottlenecked by whatever portion of the code cannot be parallelized.



Slide 14: Amdahl's Law for Multiprocessors This graph (Figure 2.4) visually demonstrates the brutal reality of Amdahl's Law.

- The x-axis shows the Number of Processors (from 1 to 1000), and the y-axis shows the total Speedup.
- The different curves represent f, which is the fraction of the software code that can actually be parallelized.
- If only 50% of your code is parallelizable ($f = 0.5$), you can add 1,000 processors, but your overall program will never run more than 2 times faster.
- Even if a massive 95% of your code is parallelizable ($f = 0.95$), the graph shows the speedup aggressively plateauing around 20x, no matter how many hundreds of processors you throw at it.



Slide 15: Little's Law This slide shifts to a different mathematical principle used in system design: Little's Law.

- **What it is:** A fundamental and incredibly simple relation with broad applications in queuing theory. It can be applied to almost any system that is statistically in a "steady state," provided there is no leakage (items dropping out of the queue).
- **The Queuing System:** Imagine a server. If the server is idle, an arriving item is served immediately; otherwise, that item joins a queue (a waiting line). This applies whether there is a single queue for a single server, a single queue for multiple servers, or multiple individual queues for multiple servers.
- **The Equation:** The law states that the average number of items in a queuing system simply equals the average rate at which items arrive, multiplied by the total average time that an item spends in the system.
- Because this relationship requires very few assumptions, its simplicity and generality make it extremely useful for engineers analyzing data flow and memory buffers.

Slide 16: System Clock This visual diagram (Figure 2.5) shows how a computer physically generates its heartbeat.

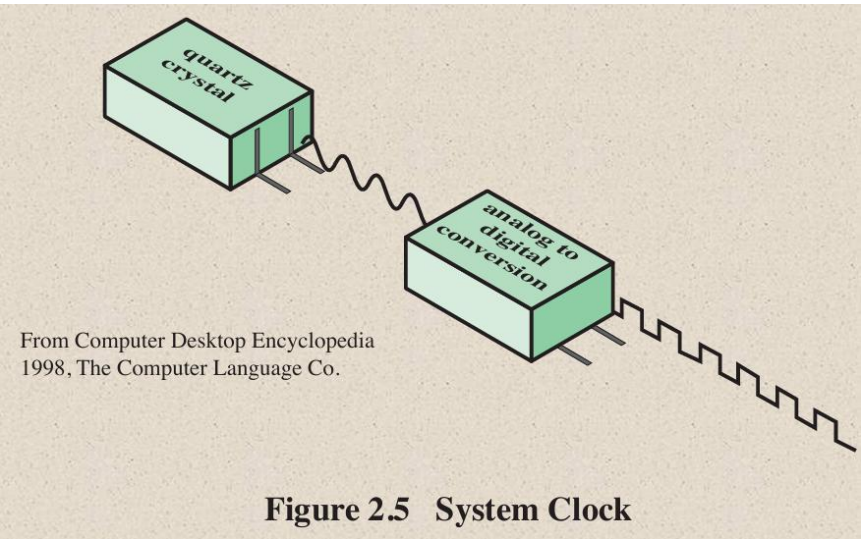


Figure 2.5 System Clock

- It starts with a physical **quartz crystal**. When electricity is applied to quartz, it vibrates at a highly precise, constant frequency.
- That continuous, wavy analog vibration is fed into an **analog to digital conversion** unit.
- The converter translates those natural waves into a rigid, digital square wave (represented by the staggered line on the right). Each pulse of this square wave acts as a single "tick" of the computer's system clock, keeping all billions of operations perfectly synchronized.

Slide 17: Performance Factors and System Attributes This table (Table 2.1) maps out how specific system design choices impact the underlying mathematical variables of performance.

	I_c	p	m	k	τ
Instruction set architecture	X	X			
Compiler technology	X	X	X		
Processor implementation		X			X
Cache and memory hierarchy				X	X

- **The Variables:**
  - I_c: Number of executed instructions.
  - p: Number of processor cycles needed to execute an instruction.
  - m: Number of memory references per instruction.
  - k: Ratio between memory and processor cycle times.

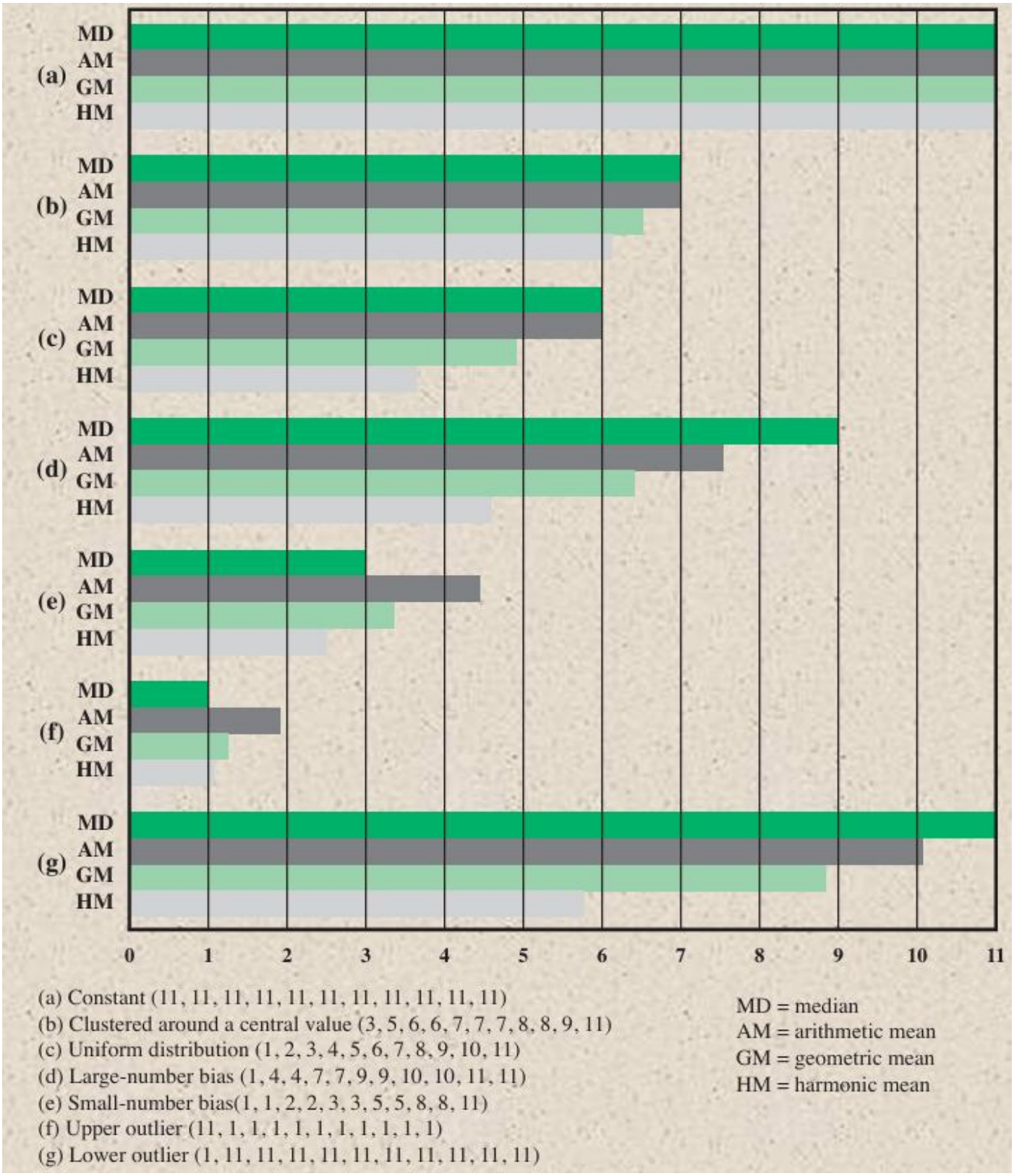
    - τ (tau): The raw processor cycle time.
- **The Attributes:** The table shows that changing the **Instruction set architecture** directly impacts I_c and p. Upgrading **Compiler technology** impacts I_c, p, and m. Changing the **Processor implementation** physically affects p and τ. Finally, altering the **Cache and memory hierarchy** affects k and τ.

Slide 18: Calculating the Mean When testing computers, engineers run multiple benchmarks and must average the results. This slide notes that comparing systems requires calculating the mean value of a set of data points related to execution time.

- The slide lists the three most common mathematical formulas used for calculating a mean in computer science: the **Arithmetic**, **Geometric**, and **Harmonic** means.

Slide 19: Comparison of Means on Various Data Sets This complex chart (Figure 2.6) proves why choosing the correct "mean" is vital, as different formulas yield drastically different results depending on the data spread.

- The chart compares the Median (MD), Arithmetic Mean (AM), Geometric Mean (GM), and Harmonic Mean (HM) across seven different data sets (a through g) where the maximum data point value is 11.
- For example, in a uniform distribution (set c), all the means line up somewhat closely.
- However, if you have a massive outlier like in set (f) (where the numbers are mostly 1s, but one is an 11), the Arithmetic Mean shoots way up, while the Harmonic Mean barely moves. This shows engineers how specific calculations can be skewed by wild benchmark results.



Slide 20: Arithmetic Mean

This slide focuses entirely on the Arithmetic Mean (AM), which is the standard "average" most people learn in school (add all numbers up, divide by the amount of numbers).

- **When to use it:** The AM is an appropriate measure if the physical sum of all the measurements is actually a meaningful and interesting value.
- **Execution Time:** The AM is a great candidate for comparing the raw execution time performance of several systems.

- **Real-world Example:** If evaluating systems for large-scale simulation studies, you would run the simulation multiple times with different inputs, and take the average execution time across all runs. Using multiple different inputs ensures the results aren't heavily biased by one weird glitch. The AM of all these runs gives a solid, reliable number for comparing system speed.
- **Proportionality:** The AM used for a time-based variable (like program execution time) has a crucial property: it is directly proportional to the total time. Simply put, if the total time doubles, the mean value doubles.

	Computer A time (secs)	Computer B time (secs)	Computer C time (secs)	Computer A rate (MFLOPS)	Computer B rate (MFLOPS)	Computer C rate (MFLOPS)
Program 1 (10^8 FP ops)	2.0	1.0	0.75	50	100	133.33
Program 2 (10^8 FP ops)	0.75	2.0	4.0	133.33	50	25
Total execution time	2.75	3.0	4.75			
Arithmetic mean of times	1.38	1.5	2.38			
Inverse of total execution time (1/sec)	0.36	0.33	0.21			
Arithmetic mean of rates				91.67	75.00	79.17
Harmonic mean of rates				72.72	66.67	42.11

Slide 21: Arithmetic and Harmonic Means for Rates

This slide presents Table 2.2, which contrasts how the Arithmetic Mean and Harmonic Mean handle performance *rates* (like millions of floating-point operations per second, or MFLOPS).

- **The Problem with Arithmetic Mean for Rates:** The table shows the execution times and MFLOPS rates for three computers (A, B, and C) running two different programs. If you calculate the Arithmetic Mean of their rates, Computer A appears to be the fastest at 91.67 MFLOPS. However, if you look at their actual total execution times, Computer A took 2.75 seconds, while Computer A is supposed to be the fastest, but wait—let's look closer.
- **The Harmonic Mean Solution:** The Arithmetic Mean can be misleading when dealing with rates. When you calculate the Harmonic Mean of those same rates, Computer A scores 72.72, Computer B scores 66.67, and Computer C scores 42.11. The Harmonic Mean accurately reflects the actual total execution time (inverse of total execution time), proving it is the mathematically correct way to average rates.

Slide 22: Arithmetic and Geometric Means for Normalized Results

Table 2.3 demonstrates the danger of using the Arithmetic Mean on *normalized* benchmark scores (where one computer is treated as the "baseline" score of 1.0).

- **(a) Normalized to Computer A:** When Computer A is the baseline (1.0), the Arithmetic Mean makes Computer C look terrible (2.85).

	Computer A time	Computer B time	Computer C time
Program 1	2.0 (1.0)	1.0 (0.5)	0.75 (0.38)
Program 2	0.75 (1.0)	2.0 (2.67)	4.0 (5.33)
Total execution time	2.75	3.0	4.75
Arithmetic mean of normalized times	1.00	1.58	2.85
Geometric mean of normalized times	1.00	1.15	1.41

- **(b) Normalized to Computer B:** When you change the baseline to Computer B, the Arithmetic Mean suddenly changes the relative rankings, making Computer A look worse (1.19). This proves the Arithmetic Mean is broken for normalized numbers because changing the baseline changes the conclusion.

	Computer A time	Computer B time	Computer C time
Program 1	2.0 (2.0)	1.0 (1.0)	0.75 (0.75)
Program 2	0.75 (0.38)	2.0 (1.0)	4.0 (2.0)
Total execution time	2.75	3.0	4.75
Arithmetic mean of normalized times	1.19	1.00	1.38
Geometric mean of normalized times	0.87	1.00	1.22

- **The Geometric Mean Solution:** Notice that whether you normalize to Computer A or Computer B, the ratio of the Geometric Means between the machines stays exactly the same. The Geometric Mean provides a consistent, reliable metric regardless of which machine is used as the baseline reference.

Slide 23: Another Comparison of Means for Normalized Results

Table 2.4 provides another extreme example of why the Geometric Mean is required for normalized results.

- In this data set, Computer A is great at Program 2, and Computer C is great at Program 1.

- When normalized to Computer A, the Arithmetic Mean says Computer C is five times slower (5.05).

	Computer A time	Computer B time	Computer C time
Program 1	2.0 (1.0)	1.0 (0.5)	0.20 (0.1)
Program 2	0.4 (1.0)	2.0 (5.0)	4.0 (10)
Total execution time	2.4	3.00	4.2
Arithmetic mean of normalized times	1.00	2.75	5.05
Geometric mean of normalized times	1.00	1.58	1.00

- When normalized to Computer B, the Arithmetic Mean says Computer A and C are tied (1.10).

	Computer A time	Computer B time	Computer C time
Program 1	2.0 (2.0)	1.0 (1.0)	0.20 (0.2)
Program 2	0.4 (0.2)	2.0 (1.0)	4.0 (2)
Total execution time	2.4	3.00	4.2
Arithmetic mean of normalized times	1.10	1.00	1.10
Geometric mean of normalized times	0.63	1.00	0.63

- Again, the Arithmetic Mean fails completely. The Geometric Mean remains consistent across both normalizations, showing that Computer A and C actually have identical overall geometric mean performance (1.00 in table a, and 0.63 in table b).

Slide 24: Benchmark Principles

Because raw clock speeds and math formulas can be misleading, the industry relies on "Benchmarks"—standardized test programs. This slide outlines the four desirable characteristics of a good benchmark program:

1. **Portability:** It must be written in a high-level language so it can be easily compiled and run across completely different types of machines (like Intel vs. ARM).
2. **Representativeness:** It must accurately represent a particular kind of real-world programming domain, such as systems programming, heavy numerical crunching, or commercial database work.
3. **Measurability:** It must be easy to measure its execution time and performance accurately.
4. **Wide Distribution:** It needs to be widely distributed and accepted by the industry so everyone is comparing "apples to apples".

Slide 25: System Performance Evaluation Corporation (SPEC)

This slide introduces the gold standard in industry benchmarking: SPEC.

- **What is a Benchmark Suite?:** Instead of running just one program, testers use a suite—a collection of high-level programs that together attempt to provide a fully representative test of a computer in a specific application area.
- **What is SPEC?:** SPEC is a massive industry consortium. It defines and maintains the best-known, most trusted collection of benchmark suites aimed at evaluating computer systems.
- **Purpose:** These standardized performance measurements are widely used across the entire computing industry for fair marketing comparisons and academic research purposes.

Slide 26: SPEC CPU2006

This slide details SPEC's most famous benchmark suite.

- **The Industry Standard:** CPU2006 is the best-known SPEC suite and the industry standard for testing processor-intensive applications.
- **The Focus:** It is specifically designed to measure performance for applications that spend almost all of their time doing heavy computation (taxing the CPU and RAM) rather than waiting on I/O (like reading from a hard drive).
- **The Programs:** To ensure a comprehensive test, it consists of 17 distinct floating-point programs (written in C, C++, and Fortran) and 12 integer programs (written in C and C++).
- **Scale:** To prevent processors from cheating the test with tiny caches, the suite contains over 3 million lines of actual code, representing the fifth generation of SPEC's processor-intensive suites.

Slide 27: SPEC CPU2006 Integer Benchmarks

This slide (Table 2.5) lists the specific real-world programs SPEC uses to test a processor's integer (whole number) calculation speed.

Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
400.perlbench	2.71	2,378	C	Programming Language	PERL programming language interpreter, applied to a set of three programs.
401.bzip2	2.68	2,472	C	Compression	General-purpose data compression with most work done in memory, rather than doing I/O.
403.gcc	2.24	1,064	C	C Compiler	Based on gcc Version 3.2, generates code for Opteron.
429.mcf	2.53	327	C	Combinatorial Optimization	Vehicle scheduling algorithm.
445.gobmk	2.91	1,603	C	Artificial Intelligence	Plays the game of Go, a simply described but deeply complex game.
456.hmmer	2.59	3,363	C	Search Gene Sequence	Protein sequence analysis using profile hidden Markov models.
458.sjeng	3.36	2,383	C	Artificial Intelligence	A highly ranked chess program that also plays several chess variants.
462.libquantum	5.76	3,555	C	Physics / Quantum Computing	Simulates a quantum computer, running Shor's polynomial-time factorization algorithm.
464.h264ref	6.15	3,731	C	Video Compression	H.264/AVC (Advanced Video Coding) Video compression.
471.omnetpp	1.74	687	C++	Discrete Event Simulation	Uses the OMNet++ discrete event simulator to model a large Ethernet campus network.
473.astar	1.95	1,200	C++	Path-finding Algorithms	Pathfinding library for 2D maps.
483.xalancbmk	1.92	1,184	C++	XML Processing	A modified version of Xalan-C++, which transforms XML documents to other document types.

- Instead of fake math tests, SPEC uses real applications.
- Examples include **400.perlbench** (a PERL programming language interpreter), **401.bzip2** (a general-purpose data compression tool that runs entirely in memory), and **403.gcc** (a C Compiler).
- It also tests complex logic and AI, using **445.gobmk** (an AI that

plays the deeply complex board game Go) and **458.sjeng** (a highly ranked chess program).
- Other tests cover vehicle scheduling optimization, DNA gene sequence searching, simulating quantum computing factorization, compressing H.264 video, and processing XML documents.

Slide 28: SPEC CPU2006 Floating-Point Benchmarks

This slide (Table 2.6) lists the programs SPEC uses to test floating-point (decimal/fractional number) performance, which is crucial for science and engineering.

Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
410.bwaves	3.78	1,176	Fortran	Fluid Dynamics	Computes 3D transonic transient laminar viscous flow.
416.gamess	5.44	5,189	Fortran	Quantum Chemistry	Quantum chemical computations.
433.milc	2.55	937	C	Physics / Quantum Chromodynamics	Simulates behavior of quarks and gluons
434.zeusmp	2.53	1,566	Fortran	Physics / CFD	Computational fluid dynamics simulation of astrophysical phenomena.
435.gromacs	1.98	1,958	C, Fortran	Biochemistry / Molecular Dynamics	Simulate Newtonian equations of motion for hundreds to millions of particles.
436.cactusADM	3.32	1,376	C, Fortran	Physics / General Relativity	Solves the Einstein evolution equations.
437.leslie3d	2.61	1,273	Fortran	Fluid Dynamics	Model fuel injection flows.
444.namd	2.23	2,483	C++	Biology / Molecular Dynamics	Simulates large biomolecular systems.
447.dealII	3.18	2,323	C++	Finite Element Analysis	Program library targeted at adaptive finite elements and error estimation.
450.soplex	2.32	703	C++	Linear Programming, Optimization	Test cases include railroad planning and military airlift models.
453.povray	1.48	940	C++	Image Ray-tracing	3D Image rendering.
454.calculix	2.29	3,04`	C, Fortran	Structural Mechanics	Finite element code for linear and nonlinear 3D structural applications.
459.GemsFDTD	2.95	1,320	Fortran	Computational Electromagnetics	Solves the Maxwell equations in 3D.
465.tonto	2.73	2,392	Fortran	Quantum Chemistry	Quantum chemistry package, adapted for crystallographic tasks.
470.lbm	3.82	1,500	C	Fluid Dynamics	Simulates incompressible fluids in 3D.
481.wrf	3.10	1,684	C, Fortran	Weather	Weather forecasting model
482.sphinx3	5.41	2,472	C	Speech recognition	Speech recognition software.

- These are incredibly heavy, real-world scientific workloads.
- Examples include **410.bwaves** and **470.lbm** (fluid dynamics simulators), **416.gamess** and **465.tonto** (quantum chemistry computations), and **433.milc** (simulating quarks and gluons in physics).
- Other intense tasks include simulating astrophysics phenomena, modeling fuel injection flows, analyzing structural mechanics, rendering 3D images with ray-tracing, and running massive weather forecasting models (**481.wrf**).

Slide 29: Terms Used in SPEC Documentation

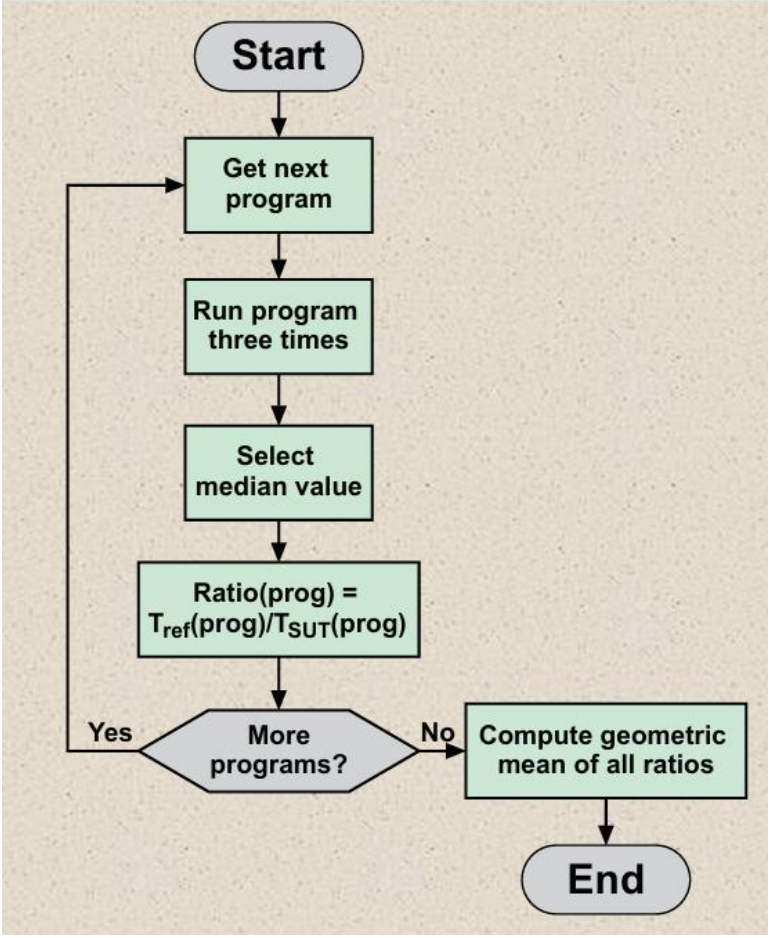
To read a SPEC scorecard, you need to understand their specific terminology.

- **System under test:** The specific computer currently being evaluated.
- **Reference machine:** A specific, standardized system used by SPEC to establish a baseline. Every benchmark is run on this machine to create a "reference time".
- **Base metric:** These results are required. They use strict, standardized guidelines for compiling the code, showing how the system performs out-of-the-box without sneaky vendor tricks.
- **Peak metric:** This allows manufacturers to aggressively optimize the compiler output specifically for their processor to show its absolute maximum theoretical potential.
- **Speed metric:** A measurement of raw time—how long it takes the computer to complete a single specific task.

- **Rate metric:** A measurement of throughput—how *many* tasks the computer can accomplish in a certain amount of time. This is crucial for multi-core processors running simultaneous workloads.

Slide 30: SPEC Evaluation Flowchart

This flowchart (Figure 2.7) shows the rigorous process of generating a SPEC score.



1. **Start:** The system gets the next benchmark program in the suite.
2. **Run:** It runs that specific program exactly three times.
3. **Median:** It selects the median execution value to throw out any weird statistical anomalies or background OS spikes.
4. **Ratio:** It calculates the performance ratio by dividing the Reference Machine's time (T_{ref}) by the System Under Test's time (T_{SUT}). (A ratio of 2.0 means it is twice as fast as the baseline).
5. **Loop:** It asks if there are more programs. If yes, it loops back to the start.
6. **End:** Once all programs are run, it computes the **Geometric Mean** of all those individual ratios to provide one final, mathematically sound benchmark score.

Slide 31: Some SPEC CINT2006 Results (Sun Blade 1000)

This slide presents real-world benchmark data (Table 2.7a) for a specific computer system, the Sun Blade 1000, running the SPEC CINT2006 integer benchmarks.

- **Execution Time Columns:** The table shows the execution time for each benchmark program run three separate times (e.g., the 400.perlbench program took 3077, 3076, and 3080 seconds). Running it three times ensures statistical reliability by weeding out random background delays.
- **Reference Time:** This is the baseline execution time established by SPEC on their standard reference machine. For 400.perlbench, the reference machine took 9770 seconds.
- **Ratio:** This is calculated by taking the reference time and dividing it by the median execution time of the tested system. For example, $9770/3077 = 3.18$. This means the Sun Blade 1000 is approximately 3.18 times faster than the baseline reference machine at running this specific Perl benchmark.

Benchmark	Execution time	Execution time	Execution time	Reference time	Ratio
400.perlbench	**3077**	3076	3080	9770	3.18
401.bzip2	**3260**	3263	3260	9650	2.96
403.gcc	2711	2701	**2702**	8050	2.98
429.mcf	2356	**2331**	2301	9120	3.91
445.gobmk	3319	**3310**	3308	10490	3.17
456.hmmer	2586	**2587**	2601	9330	3.61
458.sjeng	3452	**3449**	3449	12100	3.51
462.libquantum	**10318**	10319	10273	20720	2.01
464.h264ref	5246	5290	**5259**	22130	4.21
471.omnetpp	2565	**2572**	2582	6250	2.43
473.astar	2522	**2554**	2565	7020	2.75
483.xalancbmk	2014	2018	**2018**	6900	3.42

Slide 32: Some SPEC CINT2006 Results (Sun Blade X6250)

This slide (Table 2.7b) presents the exact same benchmark suite, but run on a much more modern and powerful system: the Sun Blade X6250.

- **Massive Performance Leap:** Looking at the execution times, the X6250 completes the 400.perlbench test in just 497 seconds, compared to the older machine's 3077 seconds.
- **Ratio Comparison:** Because of this speed, its performance ratio skyrockets to 19.66, meaning it is nearly 20 times faster than the SPEC reference machine for this task. For tasks like 462.libquantum, the ratio reaches an impressive 33.75.
- **Rate Metric:** This table introduces the "Rate" column, measuring throughput. Because modern servers have multiple cores, this metric reflects how many simultaneous tasks the system can complete over a set period. The X6250 achieves a rate of 78.63 on the Perl benchmark, showcasing its ability to handle massive, parallel workloads efficiently.

Benchmark	Execution time	Execution time	Execution time	Reference time	Ratio	Rate
400.perlbench	497	497	**497**	9770	19.66	78.63
401.bzip2	613	614	**613**	9650	15.74	62.97
403.gcc	529	**529**	529	8050	15.22	60.87
429.mcf	**472**	472	473	9120	19.32	77.29
445.gobmk	637	637	**637**	10490	16.47	65.87
456.hmmer	446	446	**446**	9330	20.92	83.68
458.sjeng	**631**	632	630	12100	19.18	76.70
462.libquantum	**614**	614	614	20720	33.75	134.98
464.h264ref	830	**830**	830	22130	26.66	106.65
471.omnetpp	619	620	**619**	6250	10.10	40.39
473.astar	580	580	**580**	7020	12.10	48.41
483.xalancbmk	**422**	422	422	6900	16.35	65.40

Slide 33: Chapter 2 Summary

This final slide recaps the core concepts explored throughout Chapter 2, giving a high-level view of performance issues in computer architecture.

- **Designing for Performance:** We examined how engineers balance cost and capacity, and the specific techniques used to increase **Microprocessor speed** (like pipelining and branch prediction).
- **Performance Balance:** We explored how engineers match the speed of the CPU with slower memory components to avoid bottlenecks.
- **Improvements in Architecture:** The chapter covered the physical and architectural shifts to smaller logic gates and larger caches, leading directly to the rise of **Multicore** processors, **MICs** (Many Integrated Cores), and **GPGPUs** (General-Purpose Graphics Processing Units) to bypass thermal limitations.
- **Mathematical Laws:** We broke down **Amdahl's Law** (calculating the maximum theoretical speedup from parallel processing) and **Little's Law** (analyzing queuing systems).
- **Measurement and Benchmarking:** Finally, we covered the **Basic measures of computer performance** (clock speed and instruction rates), the critical importance of properly **Calculating the mean** (Arithmetic vs. Harmonic vs. Geometric), and how industry-standard **SPEC benchmarks** are used to provide fair, accurate comparisons between different systems.